Bloxorz 3D

A 3D Puzzle Game Built with WebGL

Ali Mursaliyev & Zainab Soomro Computer Graphics — April 2026

OVERVIEW

Roll a block to reach the goal hole.

Three.js / WebGL — runs entirely in the browser

Custom shaders, shadow mapping, PBR materials

Quaternion-based rolling animation

Particle system, fog, tone mapping

Fall detection, fragile tiles, win / lose conditions

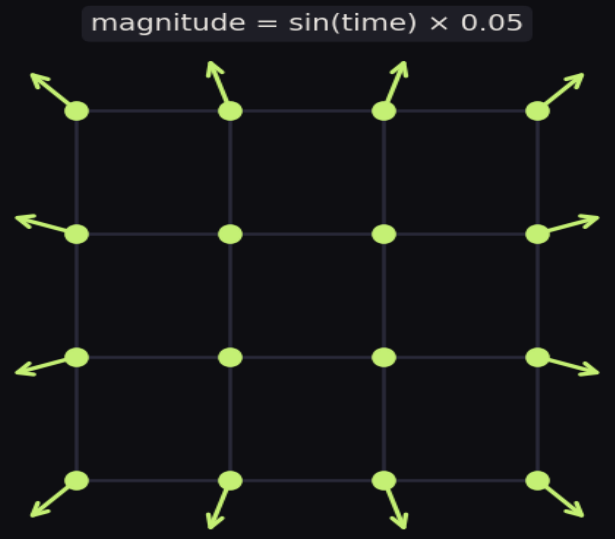
Vertex Shader

A small program that runs on every corner of the block — on the GPU

Moves each point through 3 steps:

Object space → Camera space → Screen (flat)

Breathing effect: each vertex is pushed along its normal
by a sine wave over time — the block subtly inflates and deflates



every vertex pushed outward along its normal

Fragment Shader

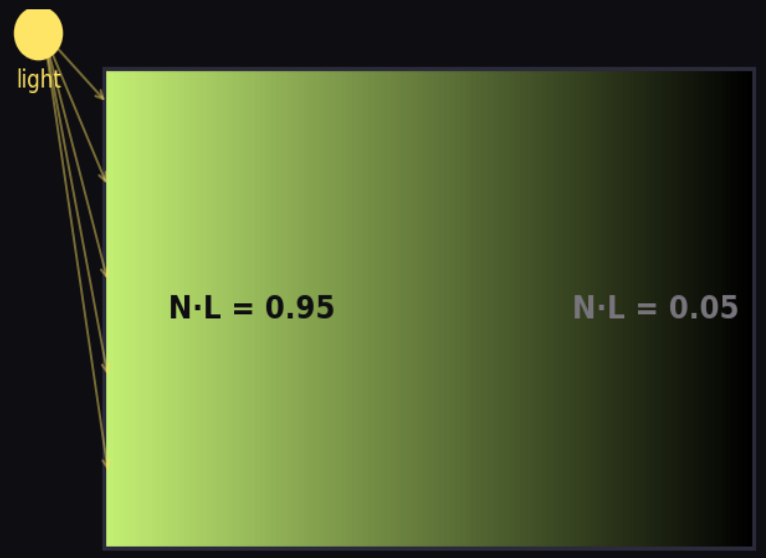
Same idea as the vertex shader — but runs on every pixel, not every corner

Colors each pixel using one rule:

dot(normal, light direction) → 0 to 1 → dark to bright

Faces pointing toward the light score high — faces pointing away score low

Plus a sine-driven emission on top for the pulsing glow



dot(surface normal, light direction) → dark to bright

SHADOW MAPPING

Two-pass technique

Pass 1 — Place a camera at the light, render, save only the distances

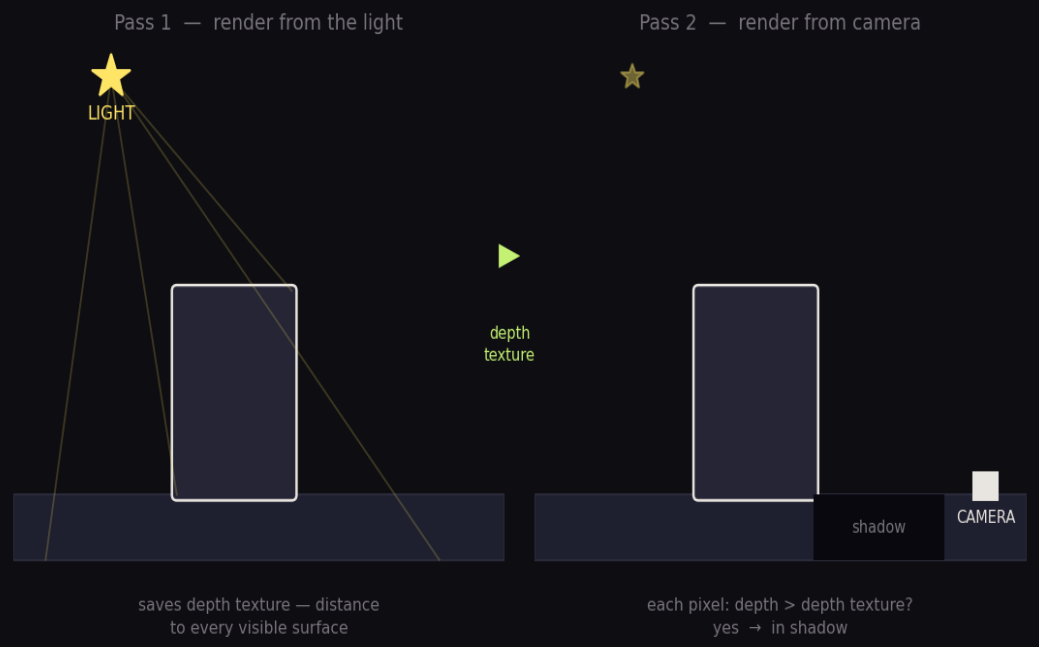
Result: a depth texture — how far the light can see at every angle

Pass 2 — Normal render from the camera

For each pixel: is this point further from the light than the map recorc

Yes → something is blocking the light → shadow

PCF — sample several nearby points in the map, average them soft



Quaternions, not Euler angles

Euler does 3 rotations in sequence — X, then Y, then Z

When the middle one hits 90°, it aligns two axes on top of each other

You lose an independent direction — gimbal lock

Quaternions store the entire rotation as one single operation

No sequence, no collapsing

SLERP — smoothly blends between start and end rotation

LERP + sine arc — smooth rolling position

Euler — gimbal lock

Y hit 90° → Y and Z now point the same direction. Axis lost.

Quaternion — single operation

one operation, no sequence
axes never collapse

MATERIALS & CAMERA

PBR + Perspective

Physically Based Rendering

Roughness — matte vs shiny
Metalness — plastic vs metal

Two numbers, many different looks

Perspective projection camera

Closer objects look bigger
Third-person, follows the block

Freezes during fall animations

VISUAL ATMOSPHERE

Visual polish

Particles — 200 floating dots
Rendered as GPU points

Fog — far tiles fade out
Creates depth and distance

Vignette — darker screen edges
Focuses attention on center

Tone mapping — ACES Filmic
Cinematic color correction

GAME MECHANICS

How the puzzle works

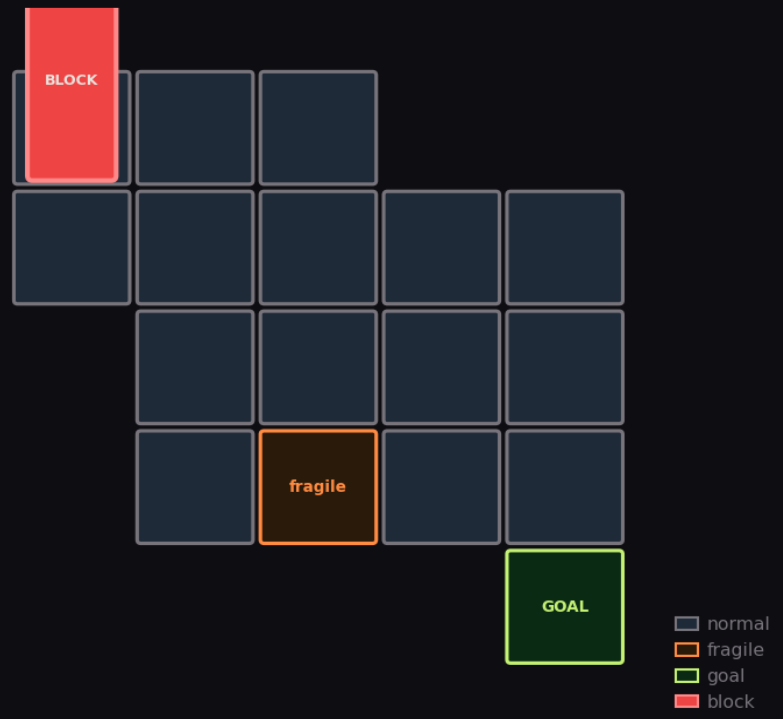
Fall detection — block checks for valid tiles after every move

Fragile tiles — break under upright block, safe when lying flat

Win condition — stand upright on the goal tile

Move counter — tracks every roll

Restart — press R anytime



WHAT'S NEXT

Remaining work

- 3–4 levels with JSON format
- Switches & bridges
- Teleporter tiles
- Texture mapping on surfaces
- Bloom post-processing
- Ambient occlusion
- Level select screen
- First-person camera

Demo